Object-Oriented Design Pattern Lab Manual

Phase 1: Requirement Analysis, 2

Phase 2: Object Identification, entity, control, boundary, 2

Phase 3: CRC Card Development, 2

Phase 4: UML Modeling, 2

Phase 5: Design Pattern Implementation: singleton patter, factory pattern, 7
Observer Pattern, Strategy Pattern,. Adapter Pattern

Object-Oriented Design Pattern Laboratory

Lab Title

Object-Oriented Analysis, Design, and Design Pattern Implementation for Real-World Software Systems

---

Lab Objective

The objective of this laboratory is to provide practical experience in analyzing real-world software problems, identifying objects, designing object interactions, modeling systems using UML, and implementing commonly used object-oriented design patterns.

After completing this laboratory, students will be able to:

- Analyze software requirements from real-world scenarios.
- Identify entity, control, and boundary objects.
- Develop CRC (Class Responsibility Collaborator) cards.
- Design UML class and sequence diagrams.
- Apply object-oriented design principles.
- Implement important design patterns using Java.

Laboratory Workflow

Students will complete the project through the following five phases:

Phase 1: Requirement Analysis

Objective

Understand the problem domain and identify the requirements of the proposed system.

Students need to analyze:

1. System Background

Describe:

- Why the system is required
- Current problems/challenges

- Expected software solution

Example:

Smart Agriculture Automation System

Traditional farming depends heavily on manual monitoring. Farmers face challenges in irrigation management, pest detection, weather uncertainty, and resource optimization. The proposed system integrates IoT sensors, AI analytics, and automated control mechanisms for intelligent farming.

---

2. System Objectives

Students should define:

- Primary objective
- Specific objectives

Example:

Primary Objective:

Develop an intelligent agriculture management platform for automated monitoring and decision support.

Specific Objectives:

- Monitor soil conditions
- Automate irrigation
- Detect crop diseases
- Provide weather-based recommendations

---

3. System Actors

Identify external users or systems interacting with the software.

Example:

Actor	Responsibility
Farmer	Monitor crops and receive alerts
Administrator	Manage system
Sensor	Provide environmental data
Weather Service	Provide weather information

Phase 2: Object Identification

Objective

Identify important objects from the problem domain.

Students classify objects into three categories:

2.1 Entity Objects

Definition

Entity objects represent real-world objects or information that the system needs to store and manage.

They usually represent the problem domain data.

Example:

Smart Agriculture System

Entity Object	Attributes
Crop	cropID, type, healthStatus
SoilSensor	sensorID, moisture, pH
IrrigationPump	pumpID, capacity, status
WeatherData	temperature, humidity

2.2 Control Objects

Definition

Control objects manage system operations, decision-making, and communication between objects.

They represent the system logic.

Example:

Control Object	Responsibility
IrrigationController	Decide irrigation timing
PestDetectionManager	Analyze pest information
HarvestPlanner	Determine harvesting time

Example:

IrrigationController

Reads SoilSensor data

↓

Checks moisture level

↓

Activates IrrigationPump

2.3 Boundary Objects

Definition

Boundary objects provide interaction between the system and external users/devices/services.

Example:

Boundary Object	Purpose
FarmerMobileApp	Farmer interaction
WeatherAPIInterface	External weather data
SensorGateway	IoT communication
AlertService	Notification delivery

Phase 3: CRC Card Development

Objective

Identify class responsibilities and relationships before UML design.

Students must prepare CRC cards for important classes.

CRC means:

- **Class**
- **Responsibilities**
- **Collaborators**

CRC Card Format

Class Name

Responsibilities

- Responsibility 1

- Responsibility 2

Collaborators

Example

CRC Card: IrrigationController

Class	IrrigationController
Responsibilities	- Monitor soil moisture - Decide irrigation requirement - Control irrigation pump
Collaborators	SoilSensor, IrrigationPump, WeatherData

Students should prepare minimum:

6 CRC Cards

for each assigned project.

Phase 4: UML Modeling

Objective

Convert object analysis into visual software design.

Students must prepare:

4.1 UML Class Diagram

The class diagram should contain:

- Class names
- Attributes
- Methods
- Relationships

Required relationships:

Association

Example:

A Farmer manages one or more Crops.

Aggregation

Example:

A Farm has many Sensors; the Sensors can exist independently of the Farm.

Composition

Example:

A House is composed of Rooms; the Rooms cannot exist without the House.

Generalization

Example:

Car and Truck are specialized types of Vehicle.

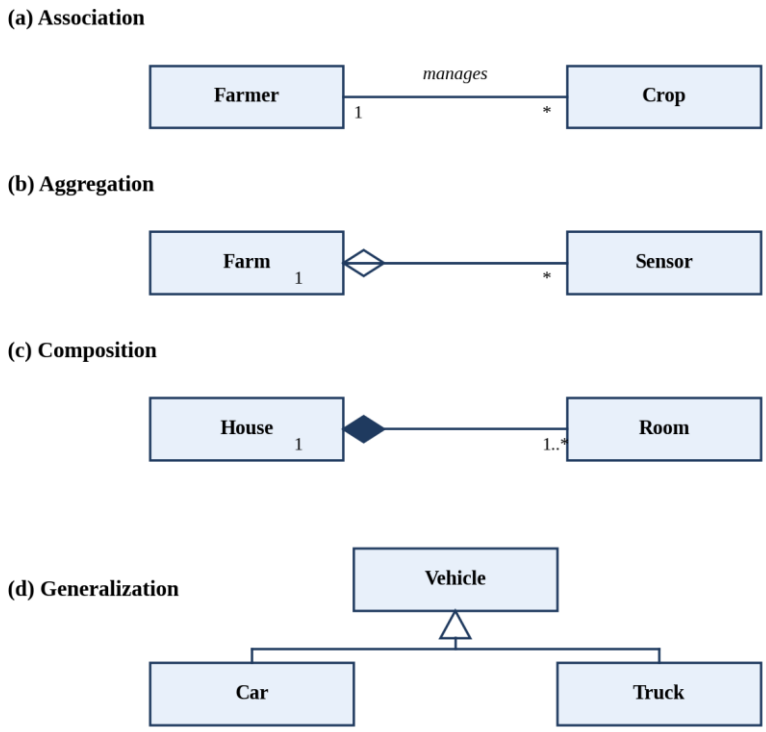


Figure 1: UML relationship notations: association, aggregation, composition, and generalization

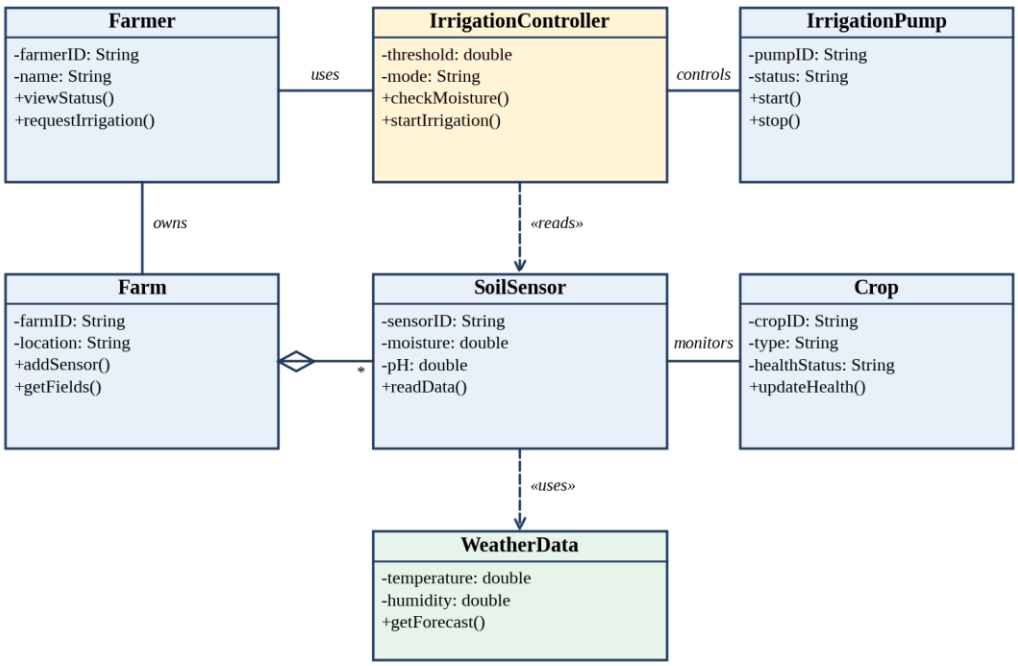


Figure 2: Sample UML class diagram for the Smart Agriculture Automation System

---

4.2 UML Sequence Diagram

Students should select one important system operation.

Example:

Smart Irrigation Workflow

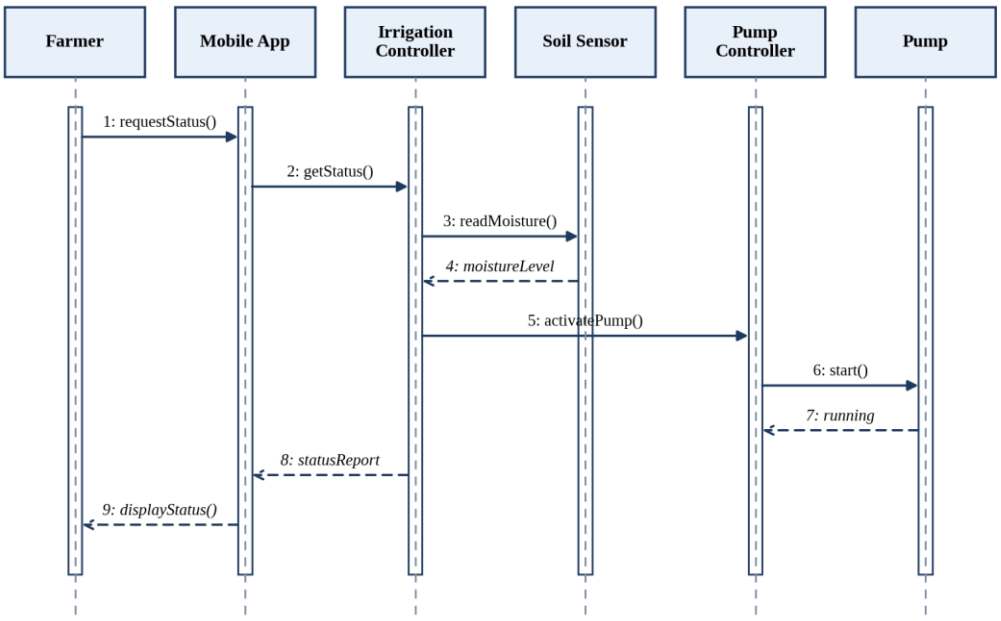


Figure 3: Sample UML sequence diagram: Smart Irrigation Workflow

The sequence diagram should show:

- Objects involved
- Message exchange
- Operation flow

Phase 5: Design Pattern Implementation

Students must implement the following five design patterns.

5.1 Singleton Pattern

Purpose

Ensure only one instance of a class exists throughout the system.

Possible Application

- Database Manager
- Configuration Manager
- System Logger

Example:

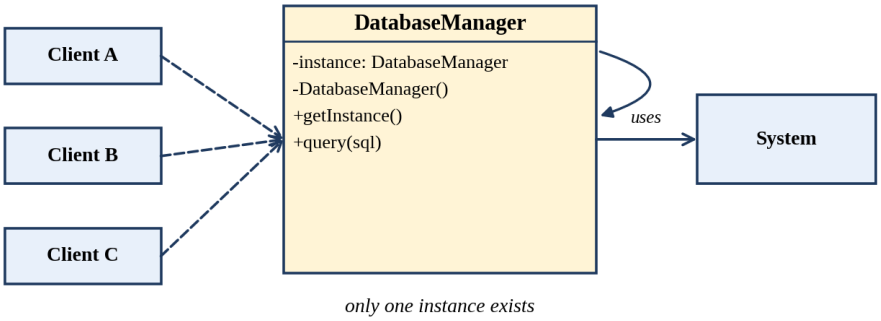


Figure 4: Singleton pattern: a single DatabaseManager instance shared by all clients

---

5.2 Factory Pattern

Purpose: Create objects dynamically without exposing object creation logic.

Possible Application

Examples:

Transportation System

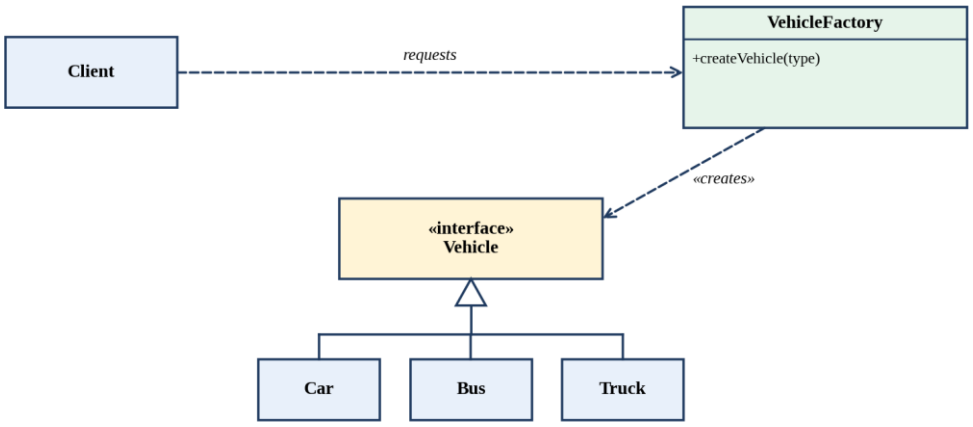


Figure 5: Factory pattern: VehicleFactory creating Car, Bus, and Truck objects

Payment System

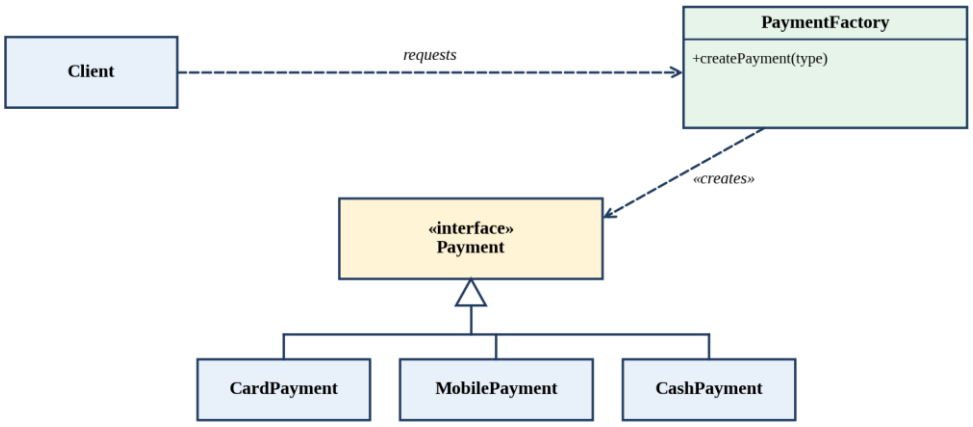


Figure 6: Factory pattern: PaymentFactory creating card, mobile, and cash payments

---

5.3 Observer Pattern

Purpose

Automatically notify dependent objects when a state changes.

Possible Applications

- Notification system
- Emergency alert
- Sensor monitoring

Example:

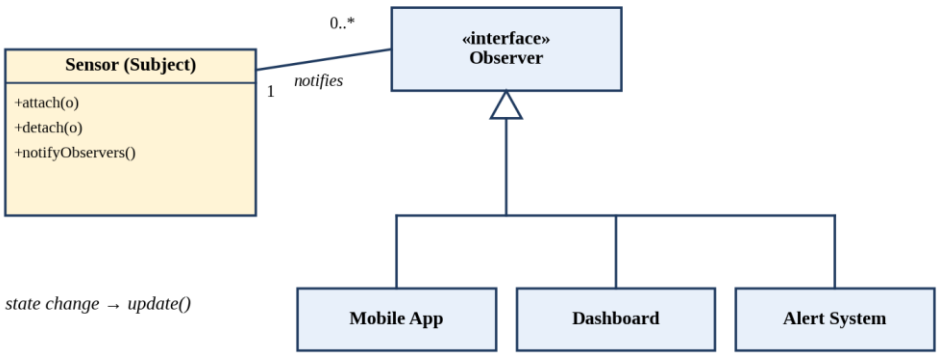


Figure 7: Observer pattern: Sensor notifying Mobile App, Dashboard, and Alert System

---

5.4 Strategy Pattern

Purpose

Allow different algorithms to be selected dynamically.

Possible Applications

Route Planning

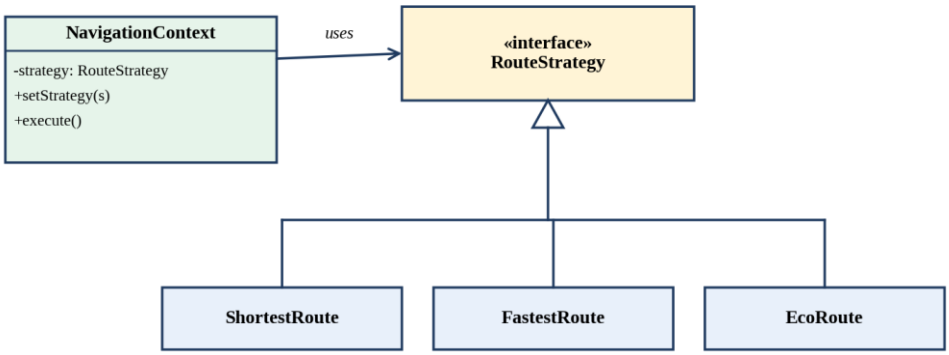


Figure 8: Strategy pattern: interchangeable route-planning algorithms

Agriculture

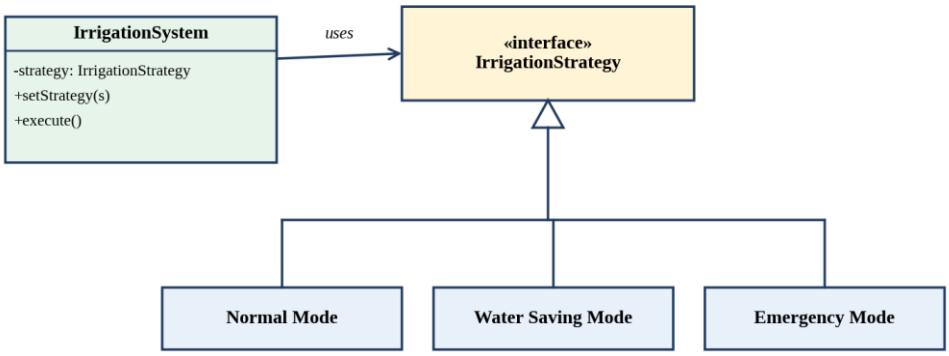


Figure 9: Strategy pattern: interchangeable irrigation modes

5.5 Adapter Pattern

Purpose

Allow incompatible systems to communicate.

Possible Applications

Weather System

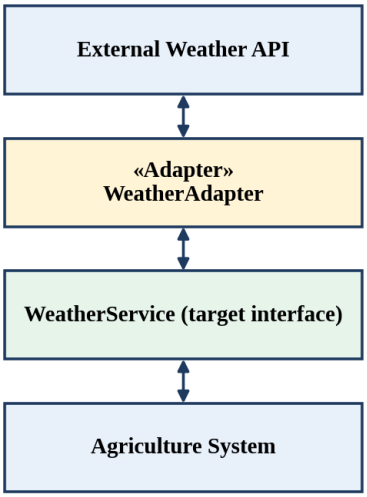


Figure 10: Adapter pattern: integrating an external weather API

Payment Gateway

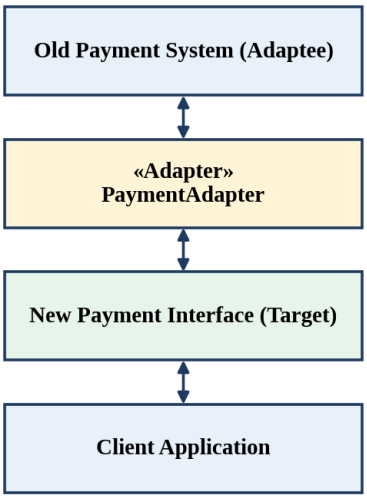


Figure 11: Adapter pattern: bridging an old payment system to a new interface

---

Final Lab Submission Format

Students must submit:

1. Project Background

2. System Objectives

3. System Actors

4. Object Identification

    4.1 Entity Objects

    4.2 Control Objects

    4.3 Boundary Objects

5. CRC Cards

6. UML Class Diagram

7. UML Sequence Diagram

8. Design Pattern Selection Explanation

9. Java Implementation

10. Testing Results

11. Conclusion

Project Proposal List

Project 1: Agricultural Automation System (AAS)

Project Overview

A smart agriculture platform that integrates IoT sensors, automation technologies, and intelligent decision-support mechanisms to improve farming efficiency.

Main Objectives

- Monitor crop and soil conditions
- Automate irrigation management
- Manage fertilizer application
- Detect pests and diseases
- Provide weather-based farming recommendations
- Support smart farming decisions

Main Actors

- Farmer
- Farm Administrator
- IoT Sensor
- Weather Service
- Drone System
- AI Analytics System

Major Entity Objects

- Crop
- Field
- SoilSensor
- WeatherData
- IrrigationPump
- FertilizerTank
- PestInformation
- Drone
- HarvestRecord

Possible Control Objects

- IrrigationController
- FertilizerScheduler
- PestDetectionManager
- HarvestPlanner
- FarmAnalyticsManager

Possible Boundary Objects

- FarmerMobileApp
- IoTSensorGateway

- WeatherAPIInterface
- DroneCameraInterface
- AlertNotificationService

Recommended Design Patterns

Pattern	Application
Singleton	Farm Data Manager
Factory	Sensor creation
Observer	Weather/Pest alerts
Strategy	Irrigation strategy
Adapter	Weather API integration

Project 2: Dhaka Urban Management System (DUMS)

Project Overview

A smart city management platform designed to integrate urban services, citizen communication, IoT monitoring, and intelligent decision support.

Main Objectives

- Manage traffic congestion
- Improve waste collection
- Handle citizen complaints
- Support emergency response
- Monitor environmental conditions
- Generate urban analytics reports

Main Actors

- Citizen
- City Administrator
- Traffic Officer
- Waste Manager
- Emergency Officer
- IoT Sensor
- Government Authority

Major Entity Objects

- Citizen

- Complaint
- TrafficSignal
- Vehicle
- WasteBin
- WasteTruck
- EmergencyRequest
- Sensor
- Location
- Report

Possible Control Objects

- TrafficController
- WasteManagementController
- EmergencyResponseManager
- ComplaintManager
- UrbanAnalyticsEngine

Possible Boundary Objects

- CitizenMobileApp
- GovernmentDashboard
- TrafficCameraInterface
- GPSInterface
- NotificationService

Recommended Design Patterns

Pattern	Application
Singleton	Central Urban Database
Factory	Urban Service Creation
Observer	Complaint Status Notification
Strategy	Route Optimization
Adapter	External Traffic API

Project 3: Smart Campus Bus Transportation System (SBTS)

Project Overview

A smart transportation platform that manages bus operations, passenger services, route optimization, and real-time monitoring.

Main Objectives

- Manage bus schedules

- Track buses using GPS
- Optimize routes
- Manage passenger tickets
- Provide real-time arrival information
- Monitor driver and vehicle performance

Main Actors

- Passenger
- Bus Driver
- Transport Manager
- Traffic Control Center
- GPS System
- Payment Service

Major Entity Objects

- Bus
- Driver
- Passenger
- Route
- BusStop
- Ticket
- Payment
- GPSLocation
- Schedule
- TrafficData

Possible Control Objects

- RouteManager
- ScheduleController
- TicketManager
- FareCalculator
- TrafficPredictionManager

Possible Boundary Objects

- PassengerMobileApp
- GPSInterface
- PaymentGateway
- TransportDashboard
- SMSNotificationService

Recommended Design Patterns

Pattern	Application
Singleton	Transport Control Center
Factory	Ticket Type Creation
Observer	Bus Arrival Notification
Strategy	Route Selection Algorithm
Adapter	Payment Gateway Integration

Project 4: Smart University Campus Management System (SUCMS)

Project Overview

An integrated campus management system that manages students, courses, classrooms, transportation, security, and digital services.

Main Objectives

- Manage student information
- Handle course registration
- Manage classrooms and schedules
- Monitor campus security
- Provide smart notifications
- Manage digital campus services

Main Actors

- Student
- Faculty Member
- Administrator
- Security Officer
- Librarian
- IoT System

Major Entity Objects

- Student
- Faculty
- Course
- Classroom
- Schedule
- LibraryBook
- Attendance
- CampusCard
- SecurityCamera

- Notice

Possible Control Objects

- CourseRegistrationManager
- AttendanceController
- SecurityMonitoringSystem
- LibraryManager
- NotificationManager

Possible Boundary Objects

- StudentPortal
- FacultyPortal
- RFIDInterface
- LibrarySystemInterface
- MobileApplication

Recommended Design Patterns

Pattern	Application
Singleton	University Configuration Manager
Factory	User Account Creation
Observer	Notice/Result Notification
Strategy	Student Evaluation Strategy
Adapter	Library External System

Project 5: Smart Campus Food Management System (SFMS)

Project Overview

A smart food management platform that manages restaurants, food ordering, delivery, inventory, and customer services.

Main Objectives

- Manage food orders
- Track inventory
- Manage restaurant operations
- Optimize delivery routes
- Provide customer notifications
- Manage payment services

Main Actors

- Customer
- Restaurant Manager

- Delivery Agent
- Food Supplier
- Payment Service
- Administrator

Major Entity Objects

- Customer
- Restaurant
- FoodItem
- Order
- DeliveryAgent
- Inventory
- Payment
- Review
- Location
- Supplier

Possible Control Objects

- OrderManager
- InventoryController
- DeliveryManager
- RecommendationEngine
- PaymentController

Possible Boundary Objects

- CustomerMobileApp
- RestaurantDashboard
- PaymentGateway
- GPSInterface
- NotificationService

Recommended Design Patterns

Pattern	Application
Singleton	Order Management System
Factory	Payment Method Creation
Observer	Order Status Notification
Strategy	Delivery Route Selection
Adapter	Third-party Payment API

Final Assignment Distribution

Group	Assigned Project
Group 1	Agricultural Automation System
Group 2	Dhaka Urban Management System
Group 3	Smart Bus Transportation System
Group 4	Smart University Campus Management System
Group 5	Smart Food Management System

These five proposals have similar complexity levels and allow students to practice the complete OOAD workflow while applying the five required design patterns.

Lab Title: Design and Implementation of Object-Oriented Design Patterns for Real-World Software Systems

Lab Objective

After completing this laboratory, students will be able to:

1. Analyze real-world problems and identify software objects.
2. Develop CRC (Class Responsibility Collaborator) cards.
3. Design UML class diagrams and interaction diagrams.
4. Select appropriate object-oriented design patterns.
5. Implement design patterns using Java.
6. Understand how design patterns improve software flexibility, reusability, and maintainability.

Lab Workflow

Students must complete the following stages for the assigned project:

Phase 1: Requirement Analysis

Students analyze the given system scenario and identify:

- System objectives
- Main actors
- Functional requirements
- Major object entities

Phase 2: Object Identification

Students identify:

Entity Objects

Objects representing real-world concepts.

Example:

Student

Book

Vehicle

Patient

Product

Control Objects

Objects responsible for business logic and coordination.

Example:

PaymentManager

BookingController

NotificationManager

Boundary Objects

Objects interacting with external users or systems.

Example:

MobileApp

WebPortal

PaymentGateway

SensorInterface

Phase 3: CRC Card Development

Students prepare minimum:

- 6 CRC cards

Each CRC card must contain:

Class Name	Responsibilities	Collaborators
Student	Register course, view result	Course, Payment
Course	Store course information	Student, Faculty

Phase 4: UML Modeling

Students prepare:

1. UML Class Diagram

Must include:

- Classes
- Attributes
- Methods
- Relationships

Relationships:

- Association
- Aggregation
- Composition
- Generalization

2. UML Sequence Diagram

Show at least one important workflow.

Example:

User

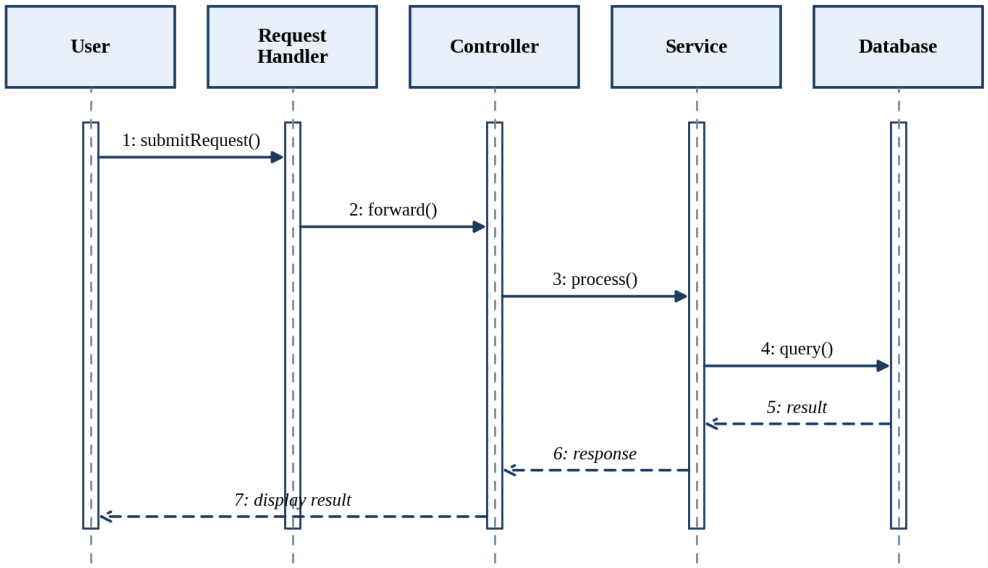


Figure 12: Sample UML sequence diagram: User request workflow

---

Phase 5: Design Pattern Implementation

Students must select and implement five design patterns from the following list:

Required Design Patterns

1. Singleton Pattern

Purpose:

Ensure only one instance of an object exists.

Possible use:

- Database connection
- Configuration manager
- System logger

Example:

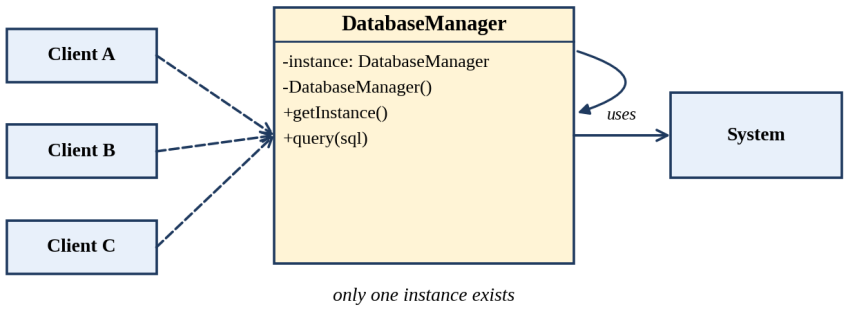


Figure 13: Singleton pattern: DatabaseManager

---

2. Factory Pattern

Purpose:

Create objects without exposing object creation logic.

Possible use:

- Vehicle creation
- Payment method creation
- User type creation

Example:

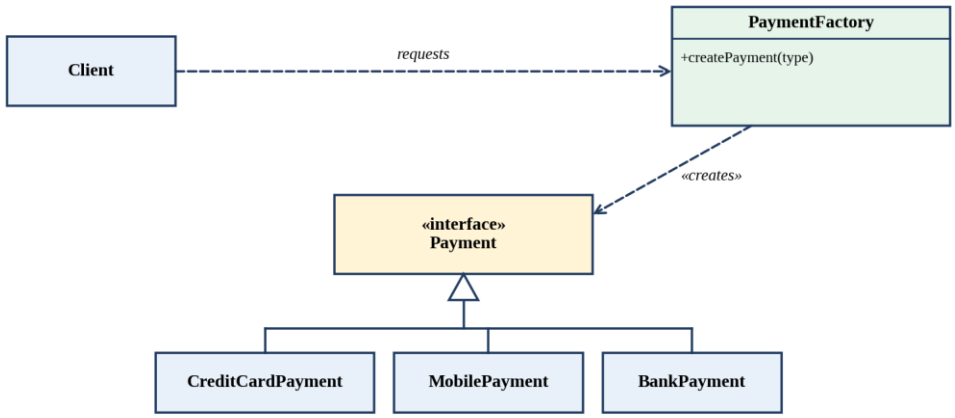


Figure 14: Factory pattern: PaymentFactory

---

3. Observer Pattern

Purpose:

Automatically notify dependent objects when a state changes.

Possible use:

- Notification system
- Alert system
- Stock monitoring

Example:

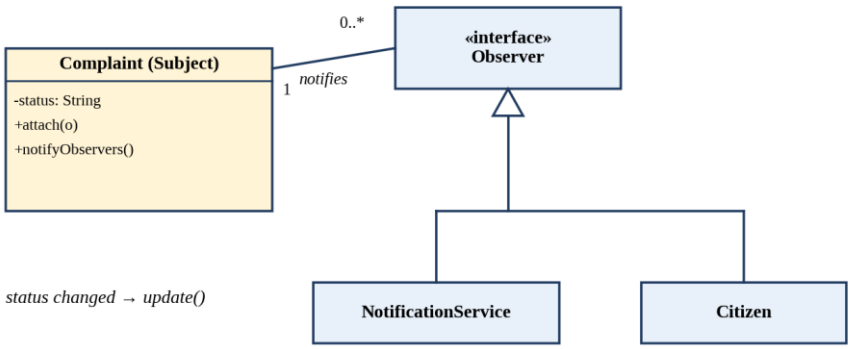


Figure 15: Observer pattern: complaint status notification

---

4. Strategy Pattern

Purpose:

Allow interchangeable algorithms.

Possible use:

- Route planning
- Sorting method
- Payment calculation
- Recommendation system

Example:

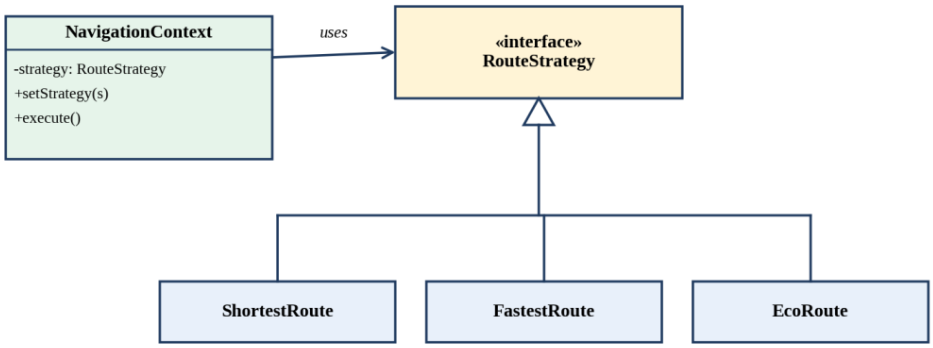


Figure 16: Strategy pattern: RouteStrategy

---

5. Adapter Pattern

Purpose:

Allow incompatible interfaces to work together.

Possible use:

- Third-party payment system
- External API integration
- Sensor connection

Example:

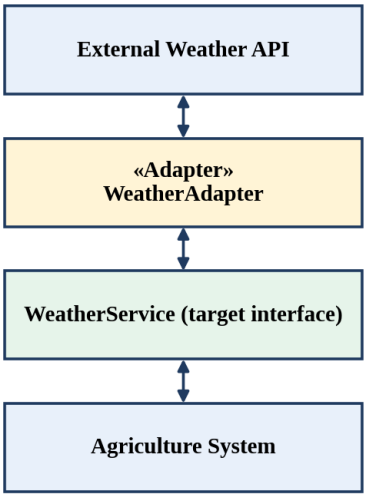


Figure 17: Adapter pattern: Weather API integration

Why avoid MVC?

You can mention in the manual:

MVC pattern is not included as a mandatory implementation pattern because it focuses mainly on software architecture separation and frequently requires database and user-interface development. This laboratory emphasizes object-oriented design patterns that demonstrate object creation, behavior management, communication, and flexibility.

Suggested Five Lab Projects

Project 1: Smart Hospital Management System (SHMS)

Background

A digital hospital platform manages patients, doctors, appointments, emergency services, and medical notifications.

Main Entities

Patient

Doctor

Appointment

MedicalRecord

Prescription

EmergencyRequest

Payment

Notification

Suggested Patterns

Pattern	Application
Singleton	Hospital database/configuration
Factory	User creation (Doctor, Patient, Staff)
Observer	Appointment notification
Strategy	Emergency priority calculation
Adapter	External laboratory system

Project 2: Online Food Delivery System

Entities

Customer

Restaurant

FoodItem

Order

DeliveryAgent

Payment

Location

Review

Patterns

Pattern	Application
Singleton	Order manager
Factory	Payment method
Observer	Order tracking notification
Strategy	Delivery route selection
Adapter	Online payment gateway

Project 3: Smart Agriculture Automation System

Entities

Crop

SoilSensor

WeatherData

IrrigationPump

Farmer

Fertilizer

Drone

Patterns

Pattern	Application
Singleton	Sensor data manager
Factory	Sensor creation
Observer	Weather alert
Strategy	Irrigation algorithm
Adapter	Weather API

Project 4: University Learning Management System (ULMS)

Entities

Student

Teacher

Course

Assignment

Exam

Result

Notification

Patterns

Pattern	Application
Singleton	Academic configuration
Factory	User account creation
Observer	Result publication notification
Strategy	Grading method
Adapter	Online exam integration

Project 5: Smart Transportation Management System

Entities

Vehicle

Driver

Route

TrafficSensor

Ticket

Payment

TrafficSignal

Patterns

Pattern	Application
Singleton	Traffic control center
Factory	Vehicle creation
Observer	Traffic alert
Strategy	Route optimization
Adapter	GPS service integration

Lab Examination Structure

Part A: Design (40 Marks)

Task	Marks
Requirement analysis	5
Object identification	10
CRC cards	10
UML class diagram	10
Sequence diagram	5

Part B: Implementation (50 Marks)

Task	Marks
Singleton implementation	10
Factory implementation	10
Observer implementation	10
Strategy implementation	10
Adapter implementation	10

Part C: Viva (10 Marks)

Sample questions:

1. Why did you select this object as an entity object?
2. Difference between aggregation and composition?
3. Why is Singleton suitable here?
4. What problem does Factory solve?
5. How does Observer reduce coupling?
6. How can Strategy improve system flexibility?

Recommended Lab Deliverable Format

Students submit:

1. Project Overview

2. Requirement Analysis

3. Actors

4. Entity-Control-Boundary Objects

5. CRC Cards

6. UML Class Diagram

7. UML Sequence Diagram

8. Design Pattern Selection Explanation

9. Java Implementation

10. Testing Results

11. Conclusion